INDIAN INSTITUTE OF TECHNOLOGY ROORKEE

Distributed Systems

Termination Detection

Recap

Chandy-Lamport's Algorithm

Cuts of a Distributed Computation

Vector Time of the cut

Termination Detection

Model

- A process can be active or idle
- only active processes send messages
- idle process can become active on receiving a **computation message**
- active process can become idle at any time
- **Control message**

Termination

- all processes are idle and no computation message are in transit

$$(\forall i :: p_i(t_f) = \text{idle}) \wedge (\forall i, j :: c_{i,j}(t_f)=0)$$

A termination detection (TD) algorithm must ensure the following:

1. Execution of a TD algorithm cannot indefinitely delay the underlying computation.
2. The termination detection algorithm must not require addition of new communication channels between processes.

Huang's Algorithm for Termination Detection by weight throwing

Assumption:

All messages are delivered in finite time (Channels are reliable).

Huang's Algorithm

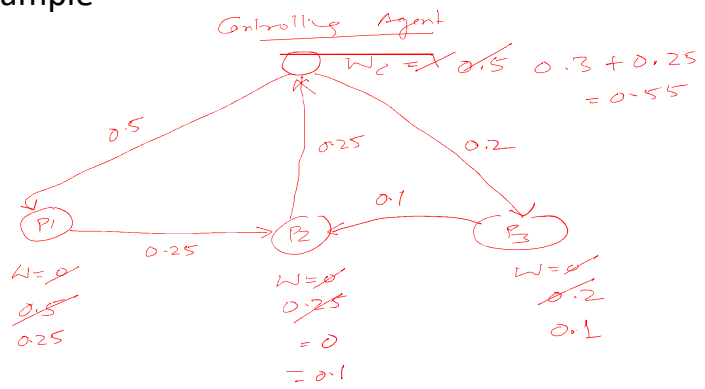
- One controlling agent, has weight 1 initially
- All other processes are idle initially and has weight 0
- Computation starts when controlling agent sends a computation message to a process
- An idle process becomes active on receiving a computation message
- B(DW) – computation message with weight DW. Can be sent only by the controlling agent or an active process
- C(DW) – control message with weight DW, sent by active processes to controlling agent when they are about to become idle

Algorithm is defined by the following 4 rules.

Let current weight at process = W

1. Send of B(DW):
   - Find W1, W2 such that W1 > 0, W2 > 0, W1 + W2 = W
   - Set W = W1 and send B(W2)
2. Receive of B(DW):
   - W += DW;
   - if idle, become active
3. Send of C(DW):
   - send C(W) to controlling agent
   - Become idle
4. Receive of C(DW):
   - W += DW
   - if W = 1, declare "termination"

An Example

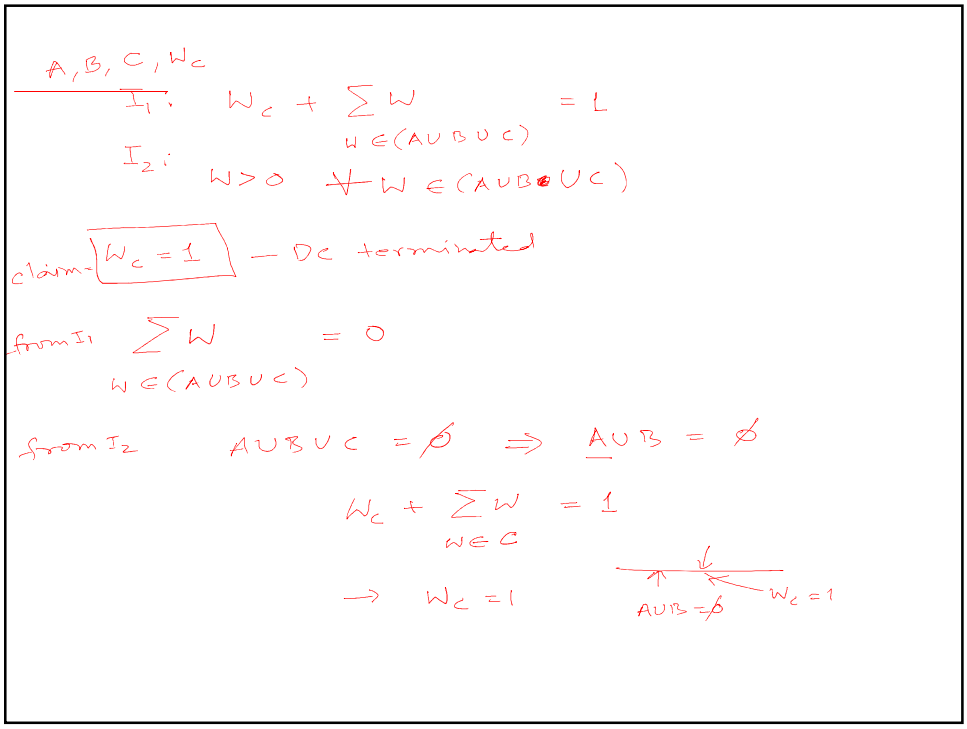


Huang's Algorithm

- Since the message delay is finite, after the computation has terminated, eventually $W_c = 1$

- Proof of Correctness
- A: The set of weight of all active processes
- B: The set of weight of all computation messages in transit
- C: The set of weights of all the control messages in transit
- W_c = Weight of the controlling agent

IIT ROORKEE



Global snapshot to detect termination

Termination of a distributed computation is a stable property.

Main idea:

- When a computation terminates, there must exist a unique process which became idle last.
- When a process goes from active to idle, it issues a request to all other processes to take a local snapshot including itself
- **When process X receives the request from Y, if it agrees that Y became idle after X, it grants the request by taking a local snapshot**
- A request is said to be *successful if all processes have taken a local snapshot for it.*

Main idea:

- If a request is successful **and** the recorded global state indicates termination of the computation, termination is detected

- Assumptions
  - Existence of logical bidirectional communication channels between every pair of process
  - Channels are reliable but non-FIFO
- Two type of messages:
  - Basic: B(x) where x is the time when message was sent
  - Control: R(x, i) is the request issued by process i at time x to other process for recording local snapshots

- Each process maintains a logical clock x
  - At any time the clock value is maximum of clock values ever received or sent in the messages
- Each process also maintains a variable k s. t. **when the process is idle**, (x, k) has the maximum value of (Dx, Dk) on all messages R(Dx, Dk) ever received or sent.

Other notations:

- p(i, t) = the state of i at time t;
- s(i → j, t) = the set of all basic messages sent from i to j by time t;
- r(i → j, t) = the set of all basic messages received by j from i by time t;
- c(i → j, u, v) = s(i → j, u) - r(i → j, v).

The algorithm is defined by the following **four rules.**

(R1): When process i is active, it may send a basic message to process j at any time by doing send a B(x) to j.

(R2): Upon receiving a B(x'), process i does

x:= max(x, x');

if (i is idle) → go active.

(R3): When process i goes idle, it does

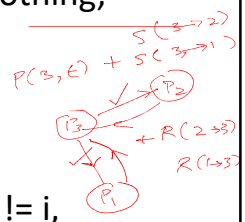
→ x:=x+1;

let k:=i;

send message R(x, k) to all other processes; take a local snapshot for the request by R(x, k).

(R4): Upon receiving message R(x', k'), process i does

- [((x', k') > (x, k)) ∧ (i is idle) → let (x, k):= (x', k'); takes a local snapshot for the request by R(x', k');
- ((x', k') <= (x, k)) ∧ (i is idle) → do nothing;
- (i is active)→ let x:=max(x', x)].



- Local snapshot on process i at time t:

  p(i, t), and **s(i → j, t)** for each i → j, j != i, and **r(k → i, t)** for each k → i, k != i.

- c(i → j, t_i, t_j): state of channel i → j recorded in global state

- The last process to terminate will have the largest clock value

  => every process will take a snapshot for it

- If the request for global snapshot is successful

- Check the recorded state for termination condition:

  $(\forall i :: p(i, t_i) = \text{idle}) \wedge (\forall i, j :: c_{i,j}(t_i, t_j)=0)$

An Implementation

- Besides x and **k,** each process also maintains a **counter** for the total number of basic messages sent minus the total number of basic messages received.
- At the start of the computation all the counters are zero and all the channels are empty.
- **y(i, t):** value of the counter on i at t
- A local snapshot of process i at t then includes p(i, t) and y(i, t).
- A global snapshot includes $p(i, t_i)$ and $y(i, t_i)$ for each i
- Computation has terminated if …

Mutual Exclusion

Mutual Exclusion

- Concurrent access to a shared resource by several uncoordinated processes requests is serialized
- very well-understood in shared memory systems
- In shared memory, solutions to mutual exclusion can be easily implemented using shared variables (e.g. semaphores)
- In distributed systems?

Critical Section Problem

Requirements:

- at most one process is in critical section (Safety property)
- if more than one requesting process, someone enters — deadlock/progress

  (selection cannot be postponed indefinitely)
- No process can prevent any other process from entering its CS indefinitely — starvation
- When no process is executing in its CS, any process that requests entry to its CS must be permitted w/o delay